Generative AI Beyond LLMs: System Implications of Multi-Modal Generation

Alicia Golden[1,2] Samuel Hsia[1,2] Fei Sun[3] Bilge Acun[1] Basil Hosmer[1] Yejin Lee[1]
Zachary DeVito[1] Jeff Johnson[1] Gu-Yeon Wei[2] David Brooks[2] Carole-Jean Wu[1]

[1]FAIR at Meta [2]Harvard University [3]Meta

arXiv:2312.14385v2 [cs.DC] 6 May 2024

***Abstract*—As the development of large-scale Generative AI models evolve beyond text (1D) generation to include image (2D) and video (3D) generation, processing spatial and temporal information presents unique challenges to quality, performance, and efficiency. We present the first work towards understanding this new system design space for multi-modal text-to-image (TTI) and text-to-video (TTV) generation models. Current model architecture designs are bifurcated into 2 categories: Diffusion- and Transformer-based models. Our systematic performance characterization on a suite of eight representative TTI/TTV models shows that after state-of-the-art optimization techniques such as Flash Attention are applied, *Convolution* accounts for up to 44% of execution time for Diffusion-based TTI models, while *Linear* layers consume up to 49% of execution time for Transformer-based models. We additionally observe that Diffusion-based TTI models resemble the `Prefill` stage of LLM inference, and benefit from 1.1-2.5x greater speedup from Flash Attention than Transformer-based TTI models that resemble the `Decode` phase. Since optimizations designed for LLMs do not map directly onto TTI/TTV models, we must conduct a thorough characterization of these workloads to gain insights for new optimization opportunities. In doing so, we define sequence length in the context of TTI/TTV models and observe sequence length can vary up to 4x in Diffusion model inference. We additionally observe temporal aspects of TTV workloads pose unique system bottlenecks, with Temporal Attention accounting for over 60% of total Attention time. Overall, our in-depth system performance characterization is a critical first step towards designing efficient and deployable systems for emerging TTI/TTV workloads.**

***Index Terms*—Generative AI, Multi-Modal, Diffusion Model, Transformer, Sequence Length, Attention**

I. Introduction

Recent advancements in generative AI have prompted immense effort into the development of efficient and scalable models for text generation [1]–[3]. The advent of Large Language Models (LLMs) has spurred myriad applications including chatbots, such as ChatGPT [4], email assistants [5], and coding tools [6]. Significant effort has gone into increasing the efficiency of these models when deployed at-scale, helping enable ChatGPT alone to serve over 100 million active users per week [7]. Yet text generation is just the tip of the iceberg. A one-dimensional representation of information lacks the ability to convey spatial and temporal information, both of which are critical for understanding the world around us. The natural progression of these large-scale generative AI models is thus to evolve from *text* (1D) to *image* (2D) to *video* (3D). However, moving to higher dimensional representations presents numerous challenges to quality, performance, and efficiency. While current systems have mainly been optimized for LLMs via techniques such as Flash Attention [8], the distinct properties of Text-To-Image (TTI) and Text-To-Video (TTV) models suggest these emerging workloads may not see equal benefits — thus requiring an in-depth analysis to identify opportunities for TTI/TTV optimization. This paper carefully examines emerging TTI/TTV workloads, and highlights attributes that greatly differ from predecessor text-based LLMs.

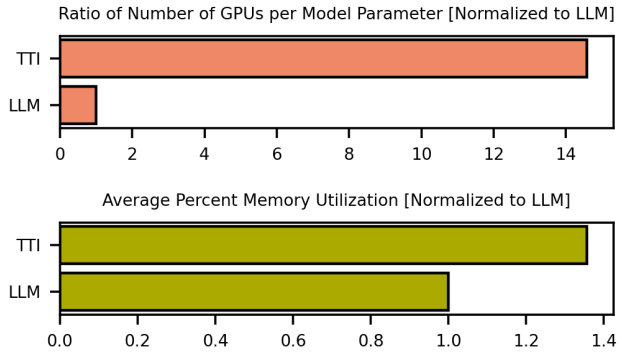

Fig. 1: Across industry-scale datacenters, Text-to-Image (TTI) models use roughly 14x more GPUs per model parameter during training and 1.35x higher memory utilization as compared to LLMs, demonstrating their growing importance at the datacenter scale.

While image and video generation models have seen significant *algorithm* advancements in recent years, relatively little has been done to optimize the deployment of these models from a *systems* perspective. New system optimizations tailored towards system performance bottleneck of TTI/TTV models have the potential to replace the generation of short video clips on the order of seconds with full movies and films. In this paper, we refer to the *performance bottleneck* as operators that dominate the execution time of the workload. Other image generation tasks such as sticker-generation [9], educational material [10], and even scientific discoveries [11], could likewise benefit from system optimizations that enable increased speed and resolution. Overcoming systems challenges is critical to enabling future applications.

To evaluate current state-of-the-art image/text generation, we first examine industry-scale generative deep learning tasks at the fleet-wide level. We find that while TTI/TTV models

are *an order of magnitude smaller* than LLMs in terms of model parameters, the number of GPUs used for training is roughly in the same order-of-magnitude. In fact, the ratio of number of GPUs per model parameter is 14x higher for TTI models than LLMs, emphasizing the importance of running these models efficiently (Figure 1). Further fleet-wide characterization reveals that this emerging class of AI workloads has distinct system requirements — average memory utilization for TTI/TTV models is roughly 35% higher than LLMs.

We subsequently take a quantitative approach to characterizing state-of-the-art TTI/TTV models, comparing the multi-dimensional design space across latency and computational intensity. We construct a model suite of eight representative text-to-image and video generation tasks and demonstrate how these models are distinct from widely-used language models, i.e., LLaMA [12]. We find that new system performance bottlenecks emerge after the application of state-of-the-art performance optimizations, such as Flash Attention [8], with *Convolution* accounting for up to 44% of execution time in Diffusion-based TTI models, and *Linear* layers consuming up to 49% of execution time in Transformer-based TTI models.

We additionally observe that traditional LLM paradigms such as *Prefill/Decode* and *Sequence Length*, do not map 1:1 onto TTI/TTV workloads. We profile sequence length over the course of inference and find that in contrast to LLMs, sequence length varies by up to 4x over the course of Diffusion model inference. In general, sequence length scales quadratically with image size in Diffusion models. Furthermore, we investigate the system performance bottleneck presented by Temporal Attention, which allows for cohesive frame generation across time in TTV models. This is in contrast to Spatial Attention, which attends across a 2D image. We find that Temporal Attention takes 2x the execution time as Spatial Attention, yet consumes 9x the FLOP count. We further observe the Temporal Attention bottleneck grows quadratically with number of frames, suggesting the need for future optimizations.

Our subsequent analysis on industry-scale generative AI use cases provides interesting insights for future system design. **TTI/TTV models exhibit the following unique system properties, which differ from traditional LLMs:**

- **High arithmetic intensity (Section II).** TTI models, and in particular diffusion-based models exhibit higher arithmetic intensity as compared to LLMs by up to 100x. This stems from the high parameter reuse that occurs during the UNet, where tens or hundreds of denoising steps cause iterations over the same number of parameters.
- **Convolution as a system performance bottleneck (Section IV-A).** After state-of-the-art optimizations such as Flash Attention are applied, the system performance bottleneck shifts to other operators such as *Convolution*, which accounts for up to 44% of execution time.
- **Unique prefill/decode correspondence (Section IV-B)**. Diffusion model inference resembles the *Prefill* phase of LLM inference, whereas Transformer-based TTI models resemble the *Decode* phase. We find that Attention kernel speedup when using Flash Attention is 1.1-2.5x greater for Diffusion models as compared to Transformer-based TTI models. This suggests that Diffusion- and Transformer-based TTI models require different optimization techniques.
- **Highly variable sequence length (Section V-A).** We profile sequence length over the course of inference and find sequence lengths in Diffusion models such as Stable Diffusion can vary by up to 4x. This variable sequence length impacts the computational intensity over the course of Diffusion model inference, and poses opportunities to tailor system design.
- **Scaling dependence with image size (Section V-B).** We find that Attention memory requirements scale as $O(L^4)$, where L is image/latent dimension. Larger image sizes see significantly higher memory requirements. Furthermore, as we increase image size to higher resolutions, we find that Convolution execution time in Diffusion models scales faster than Attention after state-of-the-art techniques such as Flash Attention are applied.
- **Temporal dimension (Section VI)**. The temporal dimension inherent in TTV models presents a new system bottleneck as compared to the TTI models from which they are built. Through profiling, we find that Temporal Attention suffers from 2x slower execution time as compared to Spatial Attention, even as it requires 9x the FLOP count. This suggests temporal attention is an important bottleneck for system design.

II. Understanding Multi-Modal Machine Learning Tasks

We first present a system-informed taxonomy of the current landscape of text-to-image (TTI) and text-to-video (TTV) generation models. Recent model development has seen a bifurcation of architectures, each characterized by their own image generation process and system characteristics. Figure 2 illustrates the image generation pipeline for these two classes of workloads: (i) Diffusion Models, including both *pixel-* and *latent-* based models, and (ii) Transformer-based Models. These same two classes of workloads additionally form the fundamental building blocks of TTV models — since video generation models typically generate a series of images (i.e., frames) using a pretrained TTI model, and then ensure the frames are temporally consistent through additional temporal attention/convolutional layers. The subsequent discussion and analysis refers to the *Inference* pipeline of image/text generation. Unlike LLMs, TTI/TTV models consist of several different model components that are trained separately and then stiched together at inference time.

A. Text-to-Image Generation Models

1) Diffusion Models: Diffusion models generate images in an iterative *denoising* process, where a random group of pixels is gradually transformed into an image that resembles a given text input [13]–[15]. During the image generation pipeline (Figure 2), a text input is first encoded into an embedding representation before being passed into the diffusion model,

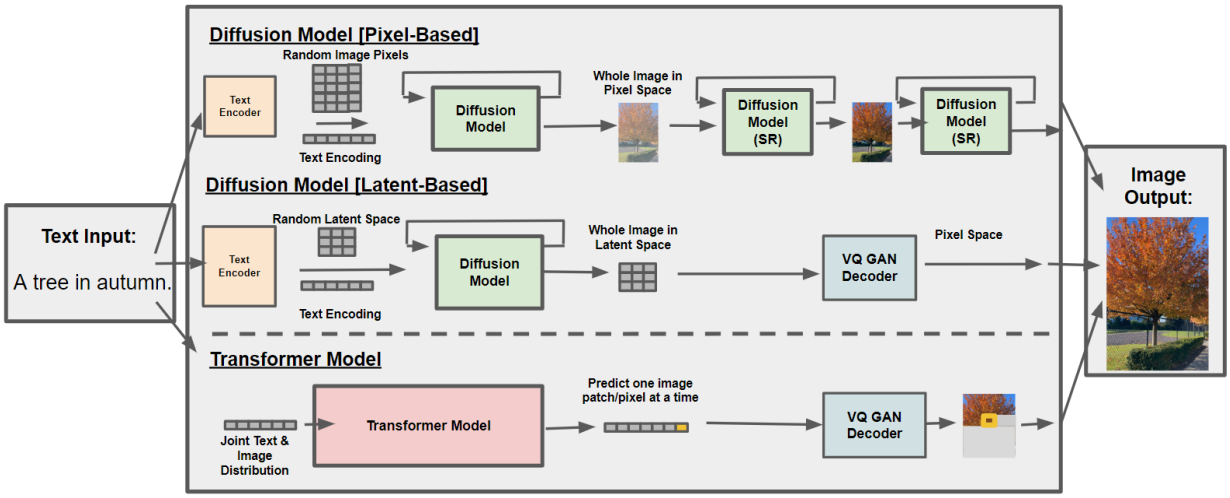

Fig. 2: Common Text-to-Image Model Architectures. Models consist of multiple independently-trained components, and are strung together during inference (shown here) to take text as input and generate an image output. Note that the top two models use a diffusion-based architectures (green), while bottom two models use transformer-based architectures (red).

along with a set of random noise pixels. As shown in Figure 3, the diffusion model architecture follows a *UNet* structure, which gradually downsamples and upsamples a noisy image through alternating *Attention* and *Resnet* blocks of various sizes [16], [17]. Note the Resnet blocks alternate with (i) **Self-Attention** blocks, which condition the generation on the image itself, and (ii) **Cross-Attention**, which attends to the text input. Before generating a final output, the image traverses through the UNet tens or hundreds of times as part of the denoising process, leading to high compute intensity, frequent parameter reuse, and long latencies (see Section II-C). Note there is an inherent trade off between number of denoising steps and image quality.

As shown in Figure 2, there are two distinct variations of diffusion models from a systems perspective — *pixel* and *latent* based. Pixel vs Latent models are distinguished by the parameter space of the diffusion model and the subsequent post-processing which is necessary once an initial image is generated. While pixel-based models operate the denoising process on standard image pixels, latent-based models transform the image into an embedding representation, making it more efficient for computation [17]. As a result, latent-based models can represent high-resolution images without the need to feed the image through subsequent SuperResolution (SR) networks as in the pixel case. This comes at the cost of a VAE or GAN-based decoder to convert latent space back to pixel space once finished.

2) Transformer Models: In contrast to the iterative nature of Diffusion models, Transformer-based TTI models generate an image sequentially. As shown in Figure 2, the transformer-based architectures model image generation as a sequence-to-sequence task, where the prediction of the next pixel (or image patch) is conditioned on all the previous patches [18]. Note that image tokens generated from the transformer model must then be decoded through a GAN decoder or equivalent to convert

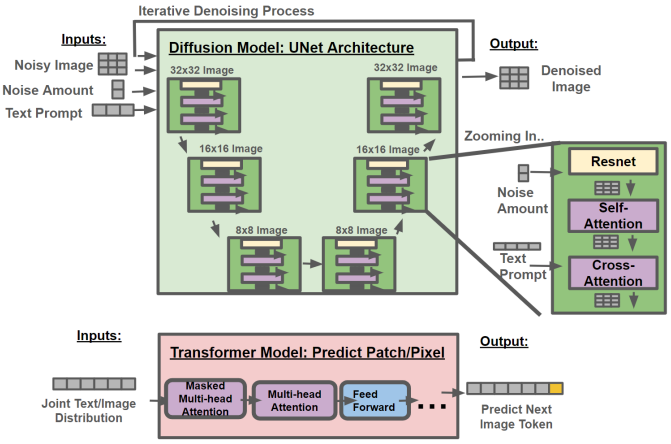

Fig. 3: Detail on Diffusion and Transformer models. Note that Diffusion models consist of Resnet blocks, Self-Attention blocks, and Cross-Attention blocks while Transformer-based models consider Self/Cross Attention and FeedForward.

back to an image representation. Figure 3 shows a detailed view of the basic transformer architecture, which consists of two multi-headed attention layers and a feedforward layer, and remains unchanged from LLMs. However, the number and arrangement of these transformer blocks varies. Compared to GPT-3, which has 96 layers and a model dimension of 12,288, Parti has 80 layers and a model dimension of 4096 [19], [20]. Other transformer-based TTI models such as Muse have 48 layers and a hidden dimension of 2048 [21].

B. Text-to-Video Models

Text-to-Video models extend traditional TTI model architectures, and often use pretrained TTI models to generate individual image frames before connecting these together with temporal attention or convolution. Like TTI models, TTV

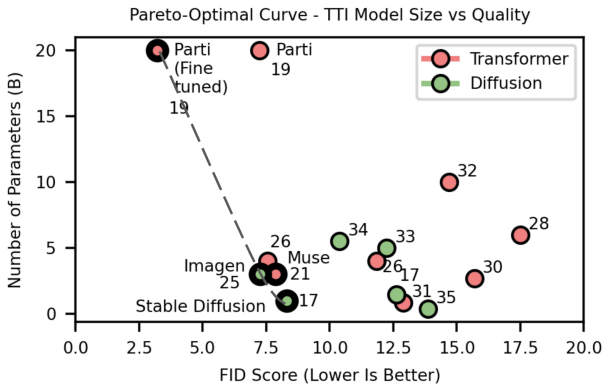

Fig. 4: Pareto-Optimal curve showing tradeoff between model quality and system resources for various Text-to-Image models. Bottom left corner is optimal. Bolded points represent models further examined in model suite outlined in Section III. Note that a corresponding figure for TTV models shows a similar trend. Diffusion TTV models often have both the lowest number of parameters and FID score.

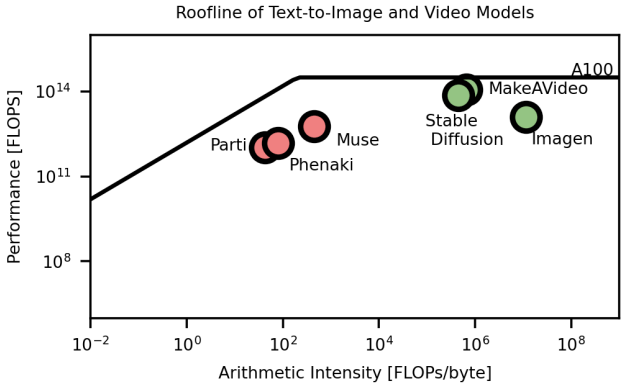

Fig. 5: Text-to-Image/Video Models Roofline on A100 GPU. Diffusion models have higher arithmetic intensity than transformer-based TTI models, and fall in the compute-bound region. Transformer-based models are memory-bandwidth bound at low batch sizes.

models can follow a diffusion-based [22] or a transformer-based model structure [23]. However, creating a temporally-cohesive video is challenging from a systems perspective. For example, **Temporal Attention** layers are often inserted after existing **Spatial Attention** layers in the UNet architecture (Figure 3), since adding an additional temporal dimension to the existing Attention call is not feasible from a memory perspective. Additionally, Attention calls are sometimes substituted for Convolutional layers to keep computational/memory costs down, especially in models with higher resolution [24].

C. System Design Space of Text-to-Image/Video Models

To further understand the system design space for emerging multi-modal generative AI technologies, Figure 4 illustrates state-of-the-art TTI generation technologies between the key design dimensions of model accuracy (x-axis) and the number of trainable parameters (y-axis). Models are labeled as their citation number [17]–[19], [21], [25]–[35]. Accuracy is measured via previously reported values of FID score [36] on the COCO dataset [37]. We omit non-open source models since the lack of publicly available implementations prevents a more in-depth system analysis. As shown, the most optimal models tend towards the bottom left corner with lowest FID score and fewest number of parameters. In general, Diffusion models tend to have higher model quality for the same number of parameters as Transformer models. However, a diverse set of models lie on the Pareto-Optimal curve, including Imagen [25] (pixel-based), Stable Diffusion [17] (latent based) and Parti (transformer), the last of which offers higher model quality than diffusion models but at 4x the parameter count. This exemplifies the importance of understanding system implications for these various architectures.

Model	Imagen [25]	Stable Diffusion [17]	Muse [21]	Parti [19]
Architecture	Diffusion (Pixel)	Diffusion (Latent)	Trans-former	Trans-former
Num Params	3B	1.45B	3B	20B
Num Layers	—	—	48	80
Model Dim	—	—	2048	4096
Attn Res	[32,16,8]	[4, 2, 1]	—	—
Text Cross Attn Res	[32,16,8]	—	—	—
Channel Mult	[1,2,4,4]	[1,2,4,4]		
Num Res Blocks	3	2	—	—
Per-Head Channels	64	8	—	—
Embed Dim	512	768	—	—
Compute	High	Medium	Low	Low
Memory	Medium	Low	Low	High
Latency	High	Medium	Low	Medium

TABLE I: Taxonomy of Text-to-Image Models

Moving beyond parameter count, we next categorize these model architectures along the major system design axes of *compute*, *memory*, and *latency*. Table I highlights specs and system requirements of four of the models from the Pareto-Optimal curve shown in Figure 4, while Figure 5 plots these models on a roofline corresponding to an A100 GPU. Note arithmetic intensity is calculated as ratio of FLOPs to required model capacity. We first find that Diffusion models have a higher arithmetic intensity than Transformer-based TTI models. This is due to the *denoising* process inherent to Diffusion models. The large number of iterations through the UNet incurs high FLOP count on a small number of parameters, leading to high parameter re-use. The denosing process also incurs high latency, given the large number of iterations required. In comparison, Transformer-based TTI models tend to be memory bound for the low batch size case shown in Figure 5. Low batch sizes are appropriate for TTI models. Transformer-based TTI require less compute and often higher memory requirements, especially in the case of Parti [19]. Yet transformer TTI models in general have faster

latencies as compared to the iterative diffusion process. This is especially true with recently introduced techniques such as parallel decoding [21].

III. Experimental Methodology

We construct a model suite of eight workloads representative of the model architectures introduced in Section II, including comparing against a state-of-the-art, publicly-available text-to-text generation model — LLaMA2 [12]. In addition to the four open-source models highlighted in Section II, we further augment our model suite to provide a realistic view of system requirements for deployment at-scale by including a production TTI model. We evaluate all models on real system hardware and measure their system requirements.

Workloads. To construct the model suite, we select representative models for each category of text-to-image model architecture (Section II). Specifically, we include models that are on the Pareto-Optimal Curve between model quality and number of parameters (see Figure 4). We select Imagen as a representative pixel-based diffusion model, given its presence on the Pareto Optimal curve. Imagen contains a T5 Encoder to encode textual information and includes three diffusion models: one to produce a 64x64 image, and two other super-resolution models that serve to upsample the image to 768x768 and 1024x1024, respectively. Additionally, to represent a latent-based model we select a model using Stable Diffusion architecture retrained on licensed data, which includes a CLIP text encoder, diffusion model, and a VQ VGAN model. For transformer-based TTI models, we select Muse and Parti to showcase the diversity of these transformer architectures, as Muse is a decoder-only transformer model that uses parallel decoding at inference time, while Parti is an encoder-decoder model that predicts image tokens autoregressively. We also include two TTV models: `Make-a-Video` [22] is built upon a diffusion model architecture, and `Phenaki` [23] that is derived from transformers.

Hardware Systems. We evaluate training and inference using NVIDIA A100 80GB GPUs. For inference analysis, we profile TTI/TTV model inference on a single GPU, since the model parameters can fit within the 80 GB memory constraints. When profiling model training, we use Fully Sharded Data Parallelism (FSDP) to train over multiple compute nodes, where each node consists of 8 A100 GPUs. Figure 6 shows how compute and communication scale as training is distributed over an increasing number of nodes.

Tools. To perform our subsequent characterization and analysis, we use PyTorch Profiler to record execution timelines. We measure GPU kernel time and annotate the model to track which CPU operators correspond to a given GPU launch kernel. We develop a profiling framework to automate this process, via inserting hooks into the forward pass. We then develop scripts to efficiently parse the resulting Pytorch Profiler output and link GPU kernels to their corresponding annotation to determine operator breakdowns, speedup, etc. We construct a similar framework to analytically calculate FLOPs. We use the NVIDIA Nsight Compute tool to examine kernel breakdowns and analyze cache hit rates, memory utilization, etc.

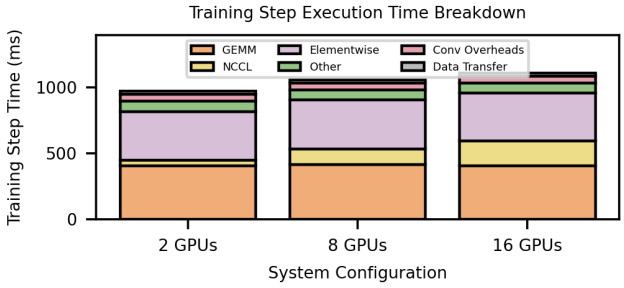

Fig. 6: Compute versus communication scaling of Stable Diffusion model as training is distributed over an increasing number of nodes (i.e., 2, 8, and 16 A100 GPUs).

IV. System Performance Characterization

Figure 7 shows the operator time breakdown (y-axis) across the model suite that was introduced in Section III. On the y-axis, we compare the execution time of the forward pass (inference) pipeline shown in Figure 2. We record the execution time breakdown of the model using baseline Attention (left bar), and plot the corresponding normalized execution time after Flash Attention V2 is applied (right bar).

A. Analyzing System Performance Bottlenecks

We first examine the execution time breakdown of baseline models (left). We observe the diversity of model operators in Diffusion-based TTI/TTV models as compared to traditional LLMs. While LLMs and transformer-based TTI models consist mainly of *Attention* and *Linear* layers, *Convolution* accounts for up to 36% of time in baseline Diffusion models. Diffusion models also have a larger variety of operators, with 4-11% of execution time attributed to *GroupNorm*. We additionally observe that *pixel-based* models spend 15% more time on convolution as compared to *latent-based* models. This is due to a higher frequency of convolution operators, since pixel-based models contain super-resolution (SR) networks that follow a UNet architecture (Figure 2), but often swap attention layers for convolution due to prohibitive memory requirements at high resolutions.

The detailed operator time characterization reveals Attention is an important system performance bottleneck in baseline models — given that Attention consumes roughly 41.3% of execution time when averaged across the TTI/TTV model suite. To accelerate the execution time of Attention, the recently proposed *Flash Attention* shows significant performance potential on GPUs [8]. The technique essentially allows for the large intermediate matrices produced in the Attention calculation to be tiled, thereby reducing memory accesses and offering significant speedup. Note that while Flash Attention was originally designed as a training optimization, previous work including the DeepSpeed Inference Framework and others has shown its benefit for inference as well [38]. Here

TABLE II: End-to-end speedup of Flash Attention as compared to Baseline Attention

LLaMA	Imagen	StableDiffusion	Muse	Parti	Prod Image	MakeAVideo	Phenaki
1.52x	1.22x	1.67x	1.11x	1.17x	1.04x	1.06x	1.15x

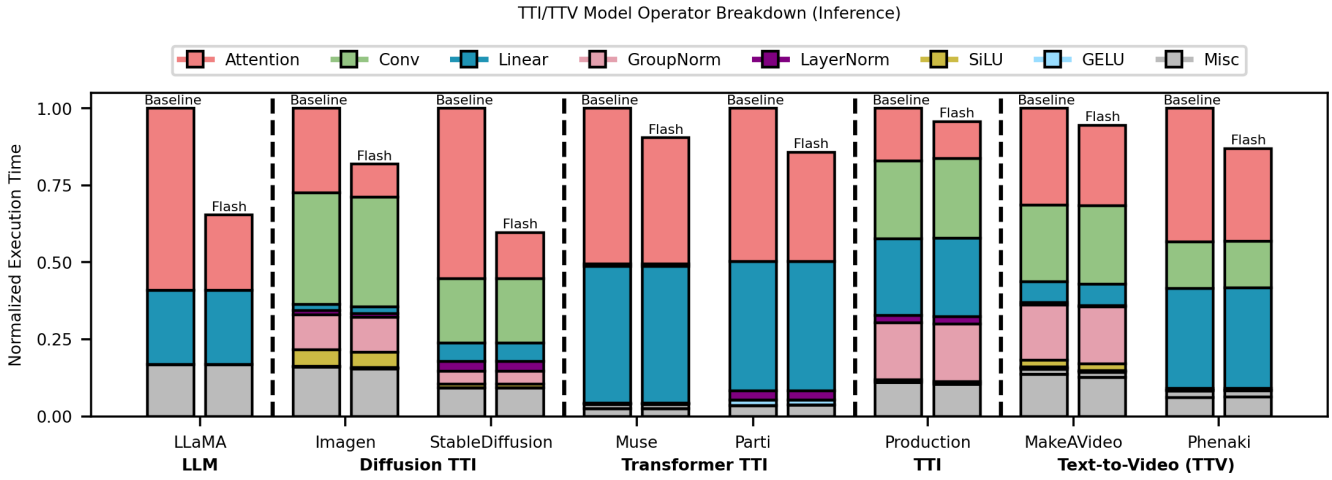

Fig. 7: Operator Breakdown Across TTI and TTV Models With Baseline Attention. First bar of each model shows model execution time with Baseline Attention, while second bar shows corresponding normalized execution time with Flash Attention.

	LLMs	**Diffusion-Based Models**	**Transformer-Based Models**
Training/ Prefill	1st token	Generate all pixels of the image at once	Process text prompt
Decode	2nd token	N/A	Generate each token autoregressively

TABLE III: Prefill/Decode for LLMs versus TTI models

we examine the impact of applying Flash Attention V2 across TTI/TTV inference workloads.

Figure 7 shows the resulting operator breakdown after Flash Attention V2 is applied, normalized to a given model's baseline execution time. Note that after Flash Attention is applied to LLaMA or transformer-based TTI models, Attention still accounts for 37-45% of total execution time. In contrast, for Diffusion models, Attention consumes only 13-25% of execution time after Flash Attention is applied, with Convolution remaining as the largest single operator block. **We find that after applying Flash Attention to Diffusion Models, the system performance bottleneck shifts to other operators such as *Convolution* instead of Attention.**

B. Evaluating Effectiveness of Flash Attention Across TTI/TTV Model Suite

We additionally observe that the end-to-end speedup of Flash Attention varies from 4-67% across the model suite. According to Amdahl's Law, the potential speedup such an optimization is impacted by two factors: (i) percent of time dedicated to Attention, and (ii) speedup of Attention module itself. While percent of time dedicated to Attention varies across model architectures, as illustrated in Section IV-A, here we focus on Attention module speedup. By examining Figure 7 and comparing the isolated speedup of the Attention module (i.e., red bar), we find that Attention module speedup from Flash Attention is 1.1-2.5x greater for Diffusion Models as compared to Transformer-based TTI models.

To understand the observed difference in Attention speedups across the model suite, we note that Attention kernel speedup varies as a factor of matrix size. We subsequently analyze TTI model speedup in the context of traditional LLM inference, which consists of two distinct phases: *Prefill* and *Decode*. *Prefill* (i.e., the initial processing of the prompt) allows for greater parallelization, as it is characterized by the processing of a large Nxd query vector, where N is sequence length and d is model dimension. A large NxN similarity matrix is created during the Attention calculation, which benefits from the tiling of Flash Attention due to reduced HBM memory accesses [8]. In contrast, the *Decode* stage generates tokens autoregressively, with queries of size 1xN. This generates a smaller Nx1 similarity matrix, which requires fewer memory accesses in the first place, and thus sees smaller speedup. Broadly speaking, *Prefill* boils down to large matrix-matrix computations while *Decode* consists of matrix-vector computations.

Table III illustrates how this concept of Prefill and Decode for LLM inference map onto TTI workloads. For diffusion-based TTI models, all pixels of an image are generated at once (see Figure 2), thus creating large matrices that resemble the prefill stage of LLM inference. This indicates Flash Attention is beneficial for Diffusion model inference. In contrast, image pixels (patches) are predicted sequentially in transformer-based TTI models due to their autoregressive nature, which

resembles decoding. Note also that transformer TTI models see less speedup from Flash Attention as compared to LLMs, due to their smaller sequence length and matrix sizes (as discussed in Section V). **Since traditional LLM paradigms such as prefill/decode do not apply in TTI/TTV workloads, this prompts a need to understand model features in the context of TTI/TTV generation in order to design optimal systems.**

V. Impact of Sequence Length

As illustrated by the varying effectiveness of Flash Attention, traditional LLM paradigms such as prefill/decode do not apply to Diffusion-based TTI and TTV workloads. In order to understand how these TTI/TTV models operate, we must translate other LLM concepts, such as sequence length, into the context of image/video generation. This will allow for more efficient system design.

LLMs are characterized by a sequence that represents the information a model can attend to, i.e., the amount of words it can refer to when generating the next subsequent word [39]. However, sequence length in state-of-the-art TTI/TTV models is directly impacted by the *image size*. In particular, the sequence length of diffusion models is proportional to $(image\ size)^2$, as attention is computed between one version of the image and the next. In transformer-based TTI models, sequence length is impacted by *image size* and text embedding size jointly.

A. Sequence Length Profiling

Figure 8 shows sequence length profiled over the course of an inference call for various TTI models. Each point on the x-axis represents a distinct time the Attention module is called during inference, while the y-axis records corresponding sequence length. Each graph is truncated to show each model's fundamental period, or the minimum repeating pattern. Note that for Parti, the sequence length linearly increases over the course of inference. This is due to the autoregessive nature of generation, where each generated token becomes part of the prompt for the next token generation. Muse uses parallel decoding instead of autoregessive generation, which is why the sequence length is constant.

In contrast, sequence lengths in Diffusion models are (i) *highly variable*, often exhibiting a cyclical pattern, and (ii) can be up to an *order of magnitude smaller* than corresponding LLMs. Note that the U-shaped nature of the sequence lengths are a result of the upsampling/downsampling blocks in the diffusion model, which change the size of the image representation (Figure 3). The width of the UNet shape is governed by the number of the convolutional blocks in the corresponding model architecture, while the height depends on the starting sequence length governed by desired output image size.

Note that we additionally quantify the memory bandwidth utilization and last level cache (LLC) miss rate for Imagen and Parti models over the same ROI as Figure 8. We find that memory bandwidth utilization and LLC miss rate follow the same pattern as the illustrated sequence length. This highlights the significance of sequence length variation over the course of model inference and its impact on the memory system.

We develop an analytical framework to model the changing memory and FLOPs requirements over the course of the forward pass of a Diffusion model. The model is built by statically examining the model computation to analyze the theoretical memory requirement. We then perform additional profiling runs in order to ensure accuracy of the model, and quantify matrix dimensions throughout model execution to verify the size of the matrices that our formulas predict.

We start with the desired image size $H_O \times W_O$, which is subsequently downsampled to latent space, $H_L \times W_L$. We define $text_encode$ as the length of the encoded text prompt. Note that sequence length squared for the *Self-Attention* blocks in the UNet is based on the size of the latent-representation, and is governed by:

$$(H_L \cdot W_L) \cdot (H_L \cdot W_L)$$

while sequence length squared for the *Cross-Attention* blocks is additionally based on text encoding as shown:

$$H_L \cdot W_L \cdot (text_encode)$$

We model memory required for the similarity matrix of one Attention calculation in the following formula below. Note that we omit batch size and assume 1 attention head for simplicity. We additionally assume FP16 (i.e. 2 bytes/param).

$$2 \cdot (H_L \cdot W_L) \cdot (H_L \cdot W_L) + 2 \cdot (H_L \cdot W_L) \cdot (text_encode)$$

$$2H_L W_L \big[H_L W_L + text_encode\big]$$

To capture the impact of sequence length variation that comes from the downsampling/upsampling convolutional blocks in the UNet, we model the downsampling factor as d^n, where d is the factor by which the image/latent representation is reduced, and n is the diffusion stage. The sequence length of a particular attention calculation depends on the stage of the UNet and the corresponding downsampling factor. This creates the periodic nature observed in Figure 8. We then have the following formula for cumulative memory requirements of the similarity matrix over the course of training:

$$2 \cdot \left[2 \sum_{n=0}^{unetdepth-1} \frac{H_L W_L}{d^{2n}} \left[\frac{H_L W_L}{d^{2n}} + text_encode \right] + \frac{H_L W_L}{d^{2unetdepth}} \left[\frac{H_L W_L}{d^{2unetdepth}} + text_encode \right] \right]$$

Our analysis reveals that since sequence length varies over the course of inference for these Diffusion-based models, memory requirements do as well. In fact, there is a quadratic relationship between latent encoding (or more generally, image size) and sequence length. Then, given the fact that memory required for the similarity matrix scales exponentially with sequence length, the relationship between latent encoding

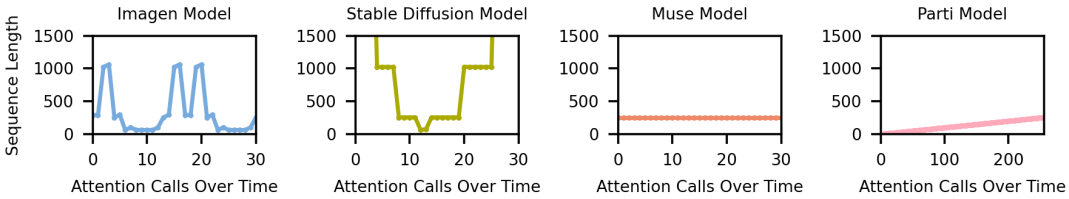

Fig. 8: Sequence length profiling across various models in model suite. Shown as sequence length over course of time. The variation in sequence length over time for diffusion models pose unique system constraints for these UNet-based models. In particular, measured values of memory bandwidth utilization and last level cache (LLC) miss rate over the identical region of interest for Imagen and Parti models reveal they correlate with sequence length variation. Note sequence length of Stable Diffusion model actually goes up to 4096, but not shown here for plotting purposes.

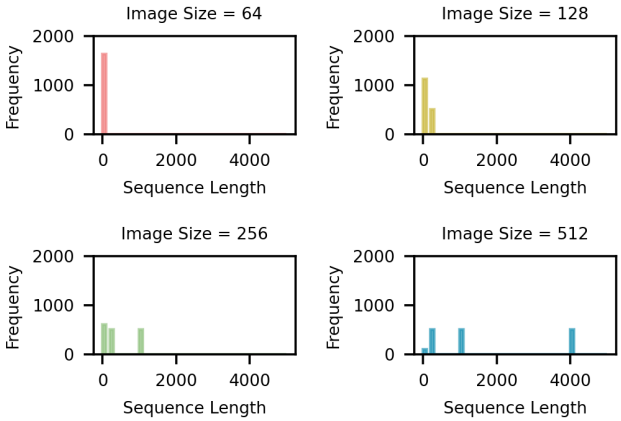

Fig. 9: Frequency distribution of sequence lengths over the course of inference for Stable Diffusion model. The value distribution shifts right for increasing image size, which corresponds to the input/output size. The distribution associated with image size of 512 corresponds to Figure 8.

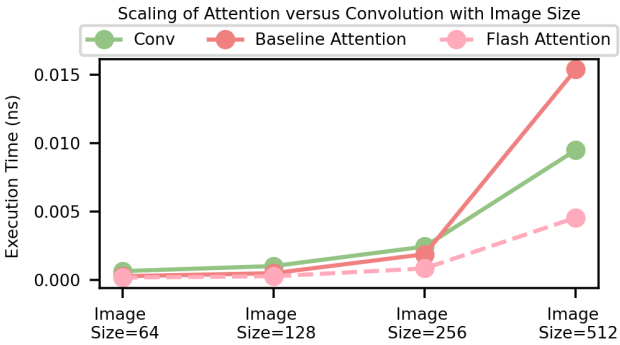

Fig. 10: Illustration of how time spent on *Attention* versus *Convolution* scales as image size increases for Stable Diffusion. Note that before Flash Attention, *Attention* execution time scales at a faster rate than Convolution execution time with increasing sequence length. However, after Flash Attention is applied, *Convolution* becomes the limiting factor.

and memory is quadruple. **As system designers, we must be aware that increasing image resolution or latent size has a $O(L^4)$relationship to memory requirements.** This presents challenges, especially for super-resolution networks that operate on high resolutions. These models often modify the UNet architecture to remove Attention since memory requirements become too large.

The relationship between sequence length and memory requirements leads to potential system optimizations. For example, different denoising steps of the diffusion process could be staggered to allow for maximum memory bandwidth utilization at any one time. Although denoising steps are traditionally sequential, certain steps could potentially be grouped together into pods to allow for this system optimization without significantly impinging on the denoising flow.

B. Implications of Scaling Image Size

We subsequently construct a case study on the Stable Diffusion model, in order to more concretely understand the impact of scaling image size (i.e., H_O, W_O). Figure 9 sweeps different input image sizes, and illustrates the corresponding sequence length distribution for Stable Diffusion inference. Note that as image size increases, sequence length distribution shifts to the right. By examining the case of a 512 x 512 image size, we note the distribution over sequence lengths is relatively equal, which confirms the symmetric nature of the sequence length profile for Stable Diffusion shown in Figure 8. As shown, the sequence lengths confine themselves to distinct buckets, which could allow future systems to tailor hardware towards sequence lengths of interest.

Additionally, we augment our *Attention* kernel analysis by comparing to the way in which *Convolution* kernels scale with image size. We again use the Stable Diffusion model as a representative case study, and sweep the desired output image size from 64 x 64 to 512 x 512, recording *Total Execution Time*. The resulting graph is shown in Figure 10. **We find that after techniques such as Flash Attention are applied, *Convolution* actually has a larger scaling dependence with image size than *Attention*.**

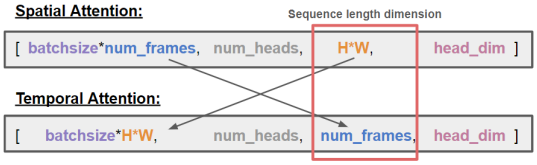

Fig. 11: Tensor dimensions are rearranged to perform Spatial versus Temporal Attention. As shown, sequence length is proportional to image size in Spatial Attention and number of frames in Temporal Attention.

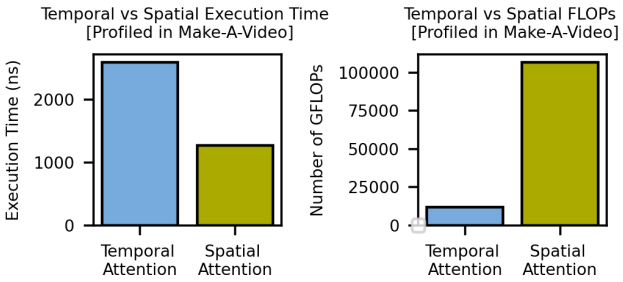

Fig. 12: Over the course of Make-A-Video inference, Temporal Attention accounts for 2x the execution time of Spatial Attention, but uses 9x less FLOPs.

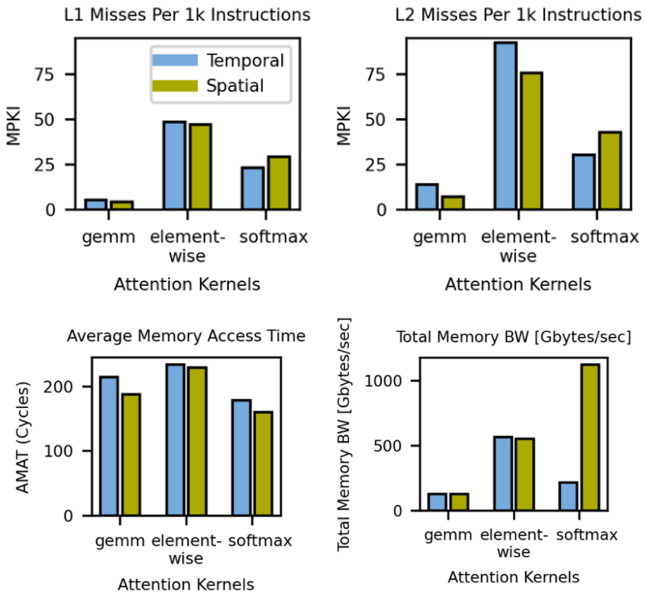

Fig. 13: Performance counters Misses Per Thousand Instructions (MPKI) for L1 and L2 Cache, Average Memory Access Time (AMAT), and Total Memory BW across Temporal and Spatial Attention kernels. Temporal Attention has 1.2-1.96x higher L2 MPKI than Spatial Attention for elementwise and gemm kernels.

VI. System Implications of Temporal Dimension

A. Temporal versus Spatial Attention

Text-to-video (TTV) models often augment their TTI backbones with temporal attention in order to connect individually-generated frames from text-to-image models. We find that this *Temporal Attention* exhibits unique system bottlenecks as compared to traditional *Spatial Attention* and explore these implications on future system design.

As shown in Figure 11, the dimensions of the Q/K/V matrices the Attention calculation [8] are reordered for the Temporal Attention, with the desired dimension to attend over shifted into the sequence length position, while other dimensions are shifted into batch size. Specifically, in Temporal Attention, the equivalent sequence length becomes the number of frames of video generation, as the goal of temporal attention is to create a cohesive video in time. In contrast, the sequence length of Spatial Attention is governed by image size.

We subsequently use the Make-A-Video model as a case study for TTV workloads, and profile the Temporal and Spatial Attention modules. We find that **Temporal Attention takes 2x the execution time of Spatial Attention (Figure 12a), despite requiring 9x fewer FLOPs (Figure 12b) when profiled in Make-A-Video.** This suggests that there exists a unique bottleneck within Temporal Attention causing this significantly longer execution time despite fewer computation.

We profile a variety of performance counters to further compare Temporal and Spatial Attention, including Misses Per Thousand Instructions (MPKI), Average Memory Access Time (AMAT), and Memory Bandwidth. As Figure 13 shows, we find that *GEMM* and *elementwise* kernels have a 1.96x and 1.22x higher L2 MPKI for Temporal attention as compared to Spatial Attention. We additionally find the AMAT for *GEMM*, *elementwise*, and *softmax* kernels is 28, 4, and 18 cycles longer for Temporal Attention, respectively. By collecting an ensemble of performance counters, we make progress towards understanding this system bottleneck, yet there is more work to be done to fully understand the nature of Temporal Attention.

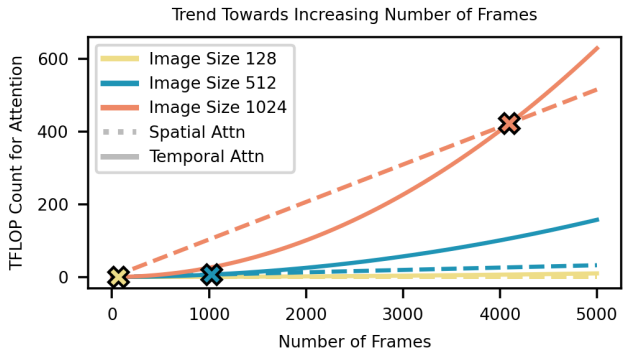

Fig. 14: Benchmark illustrating how Temporal Attention FLOPs scale quadratically with number of frames as opposed to Spatial Attention, which scales linearly.

B. Trends in Temporal Attention

Given that Temporal Attention is a system performance bottleneck, we create a benchmark based on [40] to project future compute requirements. Figure 14 sweeps the number of frames (x-axis) and measures the corresponding FLOP

count (y-axis), which is calculated by the two main matmul operations in Attention for simplicity. While FLOP count of *Spatial Attention* scales linearly with increasing number of frames, *Temporal Attention* FLOP count scales in polynomial time with equation $y = AX^2$, due to frame count being the effective sequence length. For small frame counts, *Temporal Attention* is less computationally expensive than *Spatial Attention*. However, as the number of frames increases, *Temporal Attention* becomes the dominating system bottleneck. Note that increasing image resolution prolongs the cross-over point, as *Spatial Attention* computation is highly impacted by image resolution.

We conclude by outlining that in order to design efficient systems for TTV models, we must anticipate trends towards (i) more frames, and (ii) higher resolutions. First, current videos are typically only seconds in length, with MakeAVideo generating clips of 2-3 seconds only. While frame interpolation can help extend the length of video with a given number of core frames, the generation of movies will require significantly more unique frames to represent distinct concepts. Second, we see a trend towards higher resolutions. Current TTV models stop using Attention when processing high resolutions as it is too memory intensive. This motivates the need for TTV system optimizations that enable long, coherent video generation.

VII. Related Work

Recent work has focused on characterizing the system characteristics of LLMs, and in particular, the transformer block. [1] analyzes the latency and model FLOPS utilization (MFU) to understand how to do efficient transformer inference. [41] analyzes the heterogeneity of GEMM computations in BERT. Flash Attention [8] proposes a tiling technique to reduce HBM accesses and thus improve the *Attention* bottleneck. Flash Attention V2 [42] and Flash Decoding [43] futher introduce more parallelism and inference optimizations, respectively.

Another class of work has focused on optimizing TTI/TTV models from an algorithm perspective. [3] provides an overview of recent progress in diffusion model research, and focuses on evaluating the computational efficiency of these models. [4] proposes a Multi-Architecture Multi-Expert diffusion model in order to better tailor different steps of the diffusion process to their functionality. Others propose using Retrieval Augment Generation (RAG) techniques to supplement the model architectures [30].

VIII. Conclusion

In this work, we present a detailed system characterization of an emerging class of multi-modal workloads. We find that Diffusion-based TTI models exhibit unique system bottlenecks such as *Convolution* after Flash Attention is applied. We additionally find that unlike LLMs, sequence length changes throughout the course of inference for Diffusion models, complicating the need to design systems for optimal sequence lengths rather than the largest sequence length possible. Finally, we investigate TTV models and find that temporal attention will likely become an increasing bottleneck as we mature in TTV generation. Through our in-depth system performance analysis, we take a critical first step towards designing efficient and deployable systems for emerging TTI/TTV workloads.

Acknowledgements

This research is not possible without the following colleagues at Meta. We would like to thank Uriel Singer, Adam Polyak, Yipin Zhou to understand the model requirements for text-to-image and large language models and the code base, Driss Guessous for PyTorch's Scaled Dot Product Attention implementation, and Henry Estela for his support with the training infrastructure.

References

[1] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proceedings of Machine Learning and Systems*, 2023.

[2] H. Shen, H. Chang, B. Dong, Y. Luo, and H. Meng, "Efficient llm inference on cpus," 2023.

[3] Z. Wan, X. Wang, C. Liu, S. Alam, Y. Zheng, Z. Qu, S. Yan, Y. Zhu, Q. Zhang, M. Chowdhury, and M. Zhang, "Efficient large language models: A survey," 2023.

[4] OpenAI, "Chatgpt," https://chat.openai.com/, 2023.

[5] Google, "Write with ai in google docs," 2023. [Online]. Available: https://support.google.com/docs/answer/13447609?hl=en

[6] Github, "Maximize developer velocity with ai," 2023. [Online]. Available: https://resources.github.com/copilot-for-business/

[7] A. Malik, "Openai's chatgpt now has 100 million weekly active users," 2023. [Online]. Available: https://techcrunch.com/2023/11/06/openais-chatgpt-now-has-100-million-weekly-active-users/

[8] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 16 344–16 359.

[9] Meta, "https://about.fb.com/news/2023/09/introducing-ai-powered-assistants-characters-and-creative-tools/," *Meta*, 2023. [Online]. Available: https://about.fb.com/news/2023/09/introducing-ai-powered-assistants-characters-and-creative-tools/

[10] E. Chesbrough, "Turbocharge k–12 creativity with new generative ai features in adobe express for education," *Adobe Express*, 2023. [Online]. Available: https://www.adobe.com/express/learn/blog/edu-gen-ai-features#:~:text=Students%20can%20further%20express%20their,image%20that%20matches%20their%20vision.

[11] C. Zeni, R. Pinsler, D. Zügner, A. Fowler, M. Horton, X. Fu, S. Shysheya, J. Crabbé, L. Sun, J. Smith, R. Tomioka, and T. Xie, "Mattergen: a generative model for inorganic materials design," December 2023. [Online]. Available: https://www.microsoft.com/en-us/research/publication/mattergen-a-generative-model-for-inorganic-materials-design/

[12] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open foundation and fine-tuned chat models," 2023.

[13] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 6840–6851.

[14] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," 2021.

[15] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019.

[16] O. Ronneberger, P.Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, ser. LNCS, vol. 9351. Springer, 2015, pp. 234–241, (available on arXiv:1505.04597 [cs.CV]).

[17] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022, pp. 10 684–10 695.

[18] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever, "Zero-shot text-to-image generation," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 8821–8831.

[19] J. Yu, Y. Xu, J. Y. Koh, T. Luong, G. Baid, Z. Wang, V. Vasudevan, A. Ku, Y. Yang, B. K. Ayan, B. Hutchinson, W. Han, Z. Parekh, X. Li, H. Zhang, J. Baldridge, and Y. Wu, "Scaling autoregressive models for content-rich text-to-image generation," 2022.

[20] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901.

[21] H. Chang, H. Zhang, J. Barber, A. Maschinot, J. Lezama, L. Jiang, M.-H. Yang, K. Murphy, W. T. Freeman, M. Rubinstein, Y. Li, and D. Krishnan, "Muse: Text-to-image generation via masked generative transformers," 2023.

[22] U. Singer, A. Polyak, T. Hayes, X. Yin, J. An, S. Zhang, Q. Hu, H. Yang, O. Ashual, O. Gafni, D. Parikh, S. Gupta, and Y. Taigman, "Make-a-video: Text-to-video generation without text-video data," 2022.

[23] R. Villegas, M. Babaeizadeh, P.-J. Kindermans, H. Moraldo, H. Zhang, M. T. Saffar, S. Castro, J. Kunze, and D. Erhan, "Phenaki: Variable length video generation from open domain textual description," 2022.

[24] J. Ho, W. Chan, C. Saharia, J. Whang, R. Gao, A. Gritsenko, D. P. Kingma, B. Poole, M. Norouzi, D. J. Fleet, and T. Salimans, "Imagen video: High definition video generation with diffusion models," 2022.

[25] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E. Denton, S. K. S. Ghasemipour, B. K. Ayan, S. S. Mahdavi, R. G. Lopes, T. Salimans, J. Ho, D. J. Fleet, and M. Norouzi, "Photorealistic text-to-image diffusion models with deep language understanding," 2022.

[26] O. Gafni, A. Polyak, O. Ashual, S. Sheynin, D. Parikh, and Y. Taigman, "Make-a-scene: Scene-based text-to-image generation with human priors," in *Computer Vision – ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XV*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 89–106.

[27] M. Ding, Z. Yang, W. Hong, W. Zheng, C. Zhou, D. Yin, J. Lin, X. Zou, Z. Shao, H. Yang, and J. Tang, "Cogview: Mastering text-to-image generation via transformers," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 19 822–19 835.

[28] M. Ding, W. Zheng, W. Hong, and J. Tang, "Cogview2: Faster and better text-to-image generation via hierarchical transformers," 2022.

[29] A. Aghajanyan, B. Huang, C. Ross, V. Karpukhin, H. Xu, N. Goyal, D. Okhonko, M. Joshi, G. Ghosh, M. Lewis, and L. Zettlemoyer, "Cm3: A causal masked multimodal model of the internet," 2022.

[30] M. Yasunaga, A. Aghajanyan, W. Shi, R. James, J. Leskovec, P. Liang, M. Lewis, L. Zettlemoyer, and W.-t. Yih, "Retrieval-augmented multimodal language modeling," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.

[31] C. Wu, J. Liang, L. Ji, F. Yang, Y. Fang, D. Jiang, and N. Duan, "Nüwa: Visual synthesis pre-training for neural visual world creation," 2021.

[32] Z. Feng, Z. Zhang, X. Yu, Y. Fang, L. Li, X. Chen, Y. Lu, J. Liu, W. Yin, S. Feng, Y. Sun, L. Chen, H. Tian, H. Wu, and H. Wang, "Ernie-vilg 2.0: Improving text-to-image diffusion model with knowledge-enhanced mixture-of-denoising-experts," 2023.

[33] A. Q. Nichol, P. Dhariwal, A. Ramesh, P. Shyam, P. Mishkin, B. Mcgrew, I. Sutskever, and M. Chen, "GLIDE: Towards photorealistic image generation and editing with text-guided diffusion models," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 16 784–16 804. [Online]. Available: https://proceedings.mlr.press/v162/nichol22a.html

[34] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical text-conditional image generation with clip latents," 2022.

[35] S. Gu, D. Chen, J. Bao, F. Wen, B. Zhang, D. Chen, L. Yuan, and B. Guo, "Vector quantized diffusion model for text-to-image synthesis," 2022.

[36] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/8a1d694707eb0fefe65871369074926d-Paper.pdf

[37] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft coco: Common objects in context," 2015.

[38] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, "Deepspeed-inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, ser. SC '22. IEEE Press, 2022.

[39] Z. Zheng, X. Ren, F. Xue, Y. Luo, X. Jiang, and Y. You, "Response length perception and sequence scheduling: An llm-empowered llm inference pipeline," *arXiv preprint arXiv:2305.13144*, 2023.

[40] G. Bertasius, H. Wang, and L. Torresani, "Is space-time attention all you need for video understanding?" in *Proceedings of the International Conference on Machine Learning (ICML)*, July 2021.

[41] S. Pati, S. Aga, N. Jayasena, and M. D. Sinclair, "Demystifying bert: System design implications," in *2022 IEEE International Symposium on Workload Characterization (IISWC)*, 2022, pp. 296–309.

[42] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," 2023.

[43] T. Dao, D. Haziza, F. Massa, and G. Sizov, "Flash-decoding for long-context inference," 2023. [Online]. Available: https://crfm.stanford.edu/2023/10/12/flashdecoding.html